

Assignment 2: Lenia — An Artificial Life

Assignment 1 — High Performance Computing

Plabon Shaha and Guillem Masdemont

University of Ljubljana, Spring Semester, EMAI Cohort 25–27

April 11, 2026

Exercise 1: We parallelize the algorithm and present 5 versions that keep optimizing. We compare the speedup performance with the baseline model. We consider the following 5 models:

- **GPU Naive:** An embarrassingly parallel implementation on CUDA by blocks (see `heat1.cu`).
- **GPU Shared Memory (V1):** We use `__shared__` memory to allocate the halo pixels that are not inside the block in the computation (see `heat2.cu` and `heat3.cu`).
- **GPU V2 (Constant + No Modulo):** Relocates the kernel to `__constant__` memory for hardware broadcasting and replaces expensive modulo (%) operations with `if/else` bounds checking to reduce arithmetic costs.
- **GPU V3 (FP32):** Transitions from 64-bit to 32-bit precision. By this, we unlock the GPU's dominant FP32 cores and doubles effective memory bandwidth.

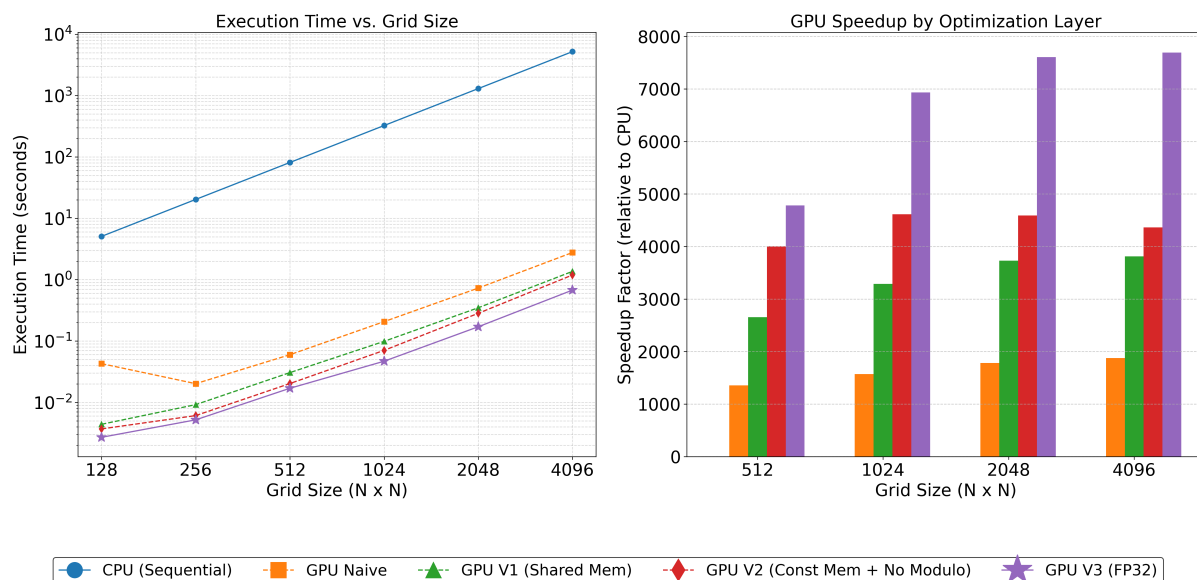


Figure 1: Performance comparison of CUDA optimization layers. **Left:** Log-log plot showing execution time versus grid size, highlighting the algebraic scaling advantage of the GPU implementations. **Right:** Grouped bar chart detailing the speedup factor of each GPU kernel relative to the sequential CPU baseline for large grid sizes.

The staggered implementation of CUDA optimizations resulted in a peak speedup of nearly 7700× over the sequential CPU baseline. The impact of each architectural consideration can be summarized as follows:

Table 1: Cross-Tile Performance Comparison for Grid Sizes 256 and 4096

N	Tile Size	T_{seq}	T_{naive}	T_{v1}	T_{v2}	$T_{v3}(f)$	Speedup(v3)
256	8	20.3207	0.0204	0.0119	0.0094	0.0078	2591.90
	16	20.3176	0.0202	0.0092	0.0061	0.0052	3876.76
	32	20.3258	0.0213	0.0094	0.0063	0.0053	3821.09
4096	8	5205.5831	2.7600	1.9797	1.4209	1.3006	4002.36
	16	5203.7136	2.7718	1.3655	1.1927	0.6765	7692.49
	32	5205.6769	2.7460	1.3746	1.2213	0.6974	7464.71

Table 2: Performance Metrics for Tile Size 8

N	T_{seq}	T_{naive}	T_{v1}	T_{v2}	$T_{v3}(f)$	Speedup(v3)
128	5.0847	0.0434	0.0048	0.0040	0.0031	1643.31
256	20.3207	0.0204	0.0119	0.0094	0.0078	2591.90
512	81.3130	0.0603	0.0417	0.0313	0.0275	2952.69
1024	325.2871	0.2123	0.1434	0.0962	0.0882	3688.49
2048	1301.3061	0.7389	0.5016	0.3584	0.3278	3969.52
4096	5205.5831	2.7600	1.9797	1.4209	1.3006	4002.36

Table 3: Performance Metrics for Tile Size 16

N	T_{seq}	T_{naive}	T_{v1}	T_{v2}	$T_{v3}(f)$	Speedup(v3)
128	5.0795	0.0428	0.0044	0.0037	0.0027	1886.26
256	20.3176	0.0202	0.0092	0.0061	0.0052	3876.76
512	81.2717	0.0599	0.0306	0.0203	0.0170	4778.02
1024	325.1207	0.2071	0.0989	0.0705	0.0469	6926.76
2048	1300.6583	0.7305	0.3488	0.2833	0.1710	7607.37
4096	5203.7136	2.7718	1.3655	1.1927	0.6765	7692.49

Table 4: Performance Metrics for Tile Size 32

N	T_{seq}	T_{naive}	T_{v1}	T_{v2}	$T_{v3}(f)$	Speedup(v3)
128	5.1617	0.0426	0.0044	0.0037	0.0027	1907.48
256	20.3258	0.0213	0.0094	0.0063	0.0053	3821.09
512	81.2846	0.0607	0.0313	0.0199	0.0173	4711.18
1024	325.0540	0.2140	0.1006	0.0732	0.0497	6537.52
2048	1300.5371	0.7232	0.3510	0.2907	0.1763	7375.32
4096	5205.6769	2.7460	1.3746	1.2213	0.6974	7464.71